



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE ESTUDIOS SUPERIORES ACATLÁN

Diplomado en Ciencia de Datos
Generación 23

Proyecto Módulo 4
Redes Neuronales Convolucionales

Aldo Alejandro Gallegos Ruiz



Índice

1. Introducción	3
1.1. Objetivo	3
1.2. Sobre el conjunto de datos	3
1.2.1. Split Train-Test	4
2. Necesidad de Negocio	5
3. Data Wrangling	6
3.1. Data Augmentation	6
3.2. Collage de imágenes	6
3.3. AEX	6
3.4. Visualización de las imágenes	7
4. Red Neuronal Convolutiva	10
4.1. Preproceso de Modelación	10
4.1.1. Generadores	10
4.1.2. Target	10
4.2. Ejecución	10
4.2.1. Capas Convolucionales, Pooling y Dropout	10
4.2.2. Flatten y Capas densamente conectadas	10
4.2.3. Callbacks	11
4.3. Entrenamiento y Validación	11
4.4. Gráfica de Resultados	12
4.4.1. Predicción	13
5. Modelo Pre-Entrenado	14
6. Comparación de Modelos de CNN	16
7. Conclusiones del Proyecto	16

1. Introducción

1.1. Objetivo

El motivo de este documento es el poder detallar y explicar los procedimientos realizados para la elaboración de este proyecto de Red Neuronal Convolutacional.

En él, se realizan técnicas y procedimientos de ingeniería de datos y análisis exploratorio vistos a lo largo del primer módulo de este diplomado. Posteriormente, se pondrán a prueba los diversos métodos de modelación vistos en este módulo IV.

1.2. Sobre el conjunto de datos

Este conjunto de datos fue proporcionado por Satellital View Inc., una empresa que se encarga de recabar capturas de imágenes satelitales de distintas infraestructuras, ya sean urbanas o rurales. La tecnología de esta empresa ha facilitado que logren trabajar de mano con el gobierno, con fines de seguridad, así como para poder identificar el avance de actividades económicas a lo largo del tiempo dentro del país.

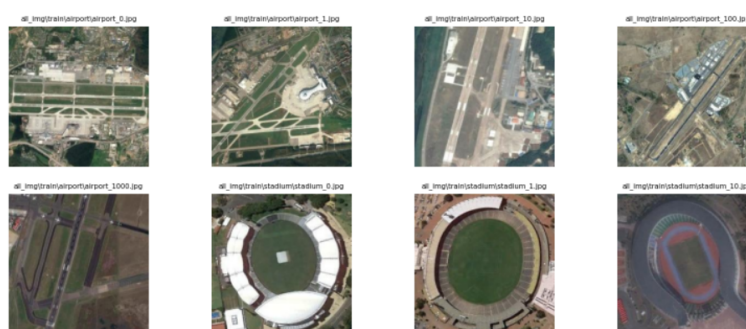
Esta empresa requiere de nuestros servicios para que nosotros elaboremos una modelo que logre identificar eficazmente qué tipo de construcción o área está siendo capturada en imagen, ya que hoy en día cuentan con un sistema bastante lento y requerir de la intervención humana. Una vez teniendo un modelo provisto por nosotros, podrán automatizar mucho este proceso de identificación de imágenes.

Contamos en total con 24 distintas bases de datos (carpetas) las cuales contiene cada una al rededor de 2000 imágenes de la construcción o área en cuestión, las cuales son las siguientes:

- Aeropuertos: 1,696
- Puentes Vehiculares: 2,000
- Áreas comerciales: 2,008
- Áreas residenciales densamente pobladas: 2,288
- Campos de cultivos: 2,009
- Carreteras: 2,008
- Canchas deportivas: 2,008
- Puertos navales: 2,009
- Áreas industriales: 1,617
- Intersecciones viales: 2,000
- Áreas de casas móviles: 2,008
- Libramientos vehiculares: 2,008
- Parques públicos: 1,232
- Estacionamientos públicos: 2,008
- Camellones: 2,040
- Vías de ferrocarril: 2,008
- Estaciones de ferrocarril: 1,784
- Glorietas: 1,640
- Embarcadero: 2,008

- Áreas residenciales poco pobladas: 1,336
- Estadios: 1,968
- Tanques de almacenamiento: 2,008
- Terrazas: 2,008
- Invernaderos: 2,104

Los datos con los que contamos es una recompilación de información de las imágenes tomadas por Satelital View Inc., nos fueron compartidas estas imágenes ya identificadas por una nomenclatura y agrupadas en carpetas según les corresponda. Lo que nosotros construiremos es un DataFrame con la ruta de la imagen, además generamos un campo llamado "label" con el nombre de la edificación o área (aeropuerto, estadio, carretera, etc.), por último un campo que identifiaca si es del conjunto de entrenamiento o test, esta segmentación la explicamos a continuación.



1.2.1. Split Train-Test

Como ya explicamos, todas nuestras imágenes están segmentadas en su correspondiente carpeta, este es nuestro universo y estructura en la que se almacenan las imágenes. Para lograr segmentar los datos para el entrenamiento y validación, nos ayudamos de la librería `shutil` de Python para que para cada que se cree una carpeta de train y test, las cuales conservarán la misma estructura original pero la carpeta train tendrá el 80% de las imágenes, mientras que test tendrá el 20% de esta.

Vale la pena mencionar que la manipulación de datos que se realizará más adelante solo será aplicado al conjunto de train.

	path	label	set
0	C:/Users/aldol/Programming/Diplo/Modulo_4/Datos_Modulo_4/proyecto/collage/test/airport/collage_airport_1006.jpg	airport	test
1	C:/Users/aldol/Programming/Diplo/Modulo_4/Datos_Modulo_4/proyecto/collage/test/airport/collage_airport_1007.jpg	airport	test
2	C:/Users/aldol/Programming/Diplo/Modulo_4/Datos_Modulo_4/proyecto/collage/test/airport/collage_airport_1008.jpg	airport	test
3	C:/Users/aldol/Programming/Diplo/Modulo_4/Datos_Modulo_4/proyecto/collage/test/airport/collage_airport_1009.jpg	airport	test
4	C:/Users/aldol/Programming/Diplo/Modulo_4/Datos_Modulo_4/proyecto/collage/test/airport/collage_airport_1014.jpg	airport	test
...	...	...	...
45800	C:/Users/aldol/Programming/Diplo/Modulo_4/Datos_Modulo_4/proyecto/collage/train/vegetable/collage_vegetable_greenhouse_995.jpg	vegetable	train
45801	C:/Users/aldol/Programming/Diplo/Modulo_4/Datos_Modulo_4/proyecto/collage/train/vegetable/collage_vegetable_greenhouse_996.jpg	vegetable	train
45802	C:/Users/aldol/Programming/Diplo/Modulo_4/Datos_Modulo_4/proyecto/collage/train/vegetable/collage_vegetable_greenhouse_997.jpg	vegetable	train
45803	C:/Users/aldol/Programming/Diplo/Modulo_4/Datos_Modulo_4/proyecto/collage/train/vegetable/collage_vegetable_greenhouse_998.jpg	vegetable	train
45804	C:/Users/aldol/Programming/Diplo/Modulo_4/Datos_Modulo_4/proyecto/collage/train/vegetable/collage_vegetable_greenhouse_999.jpg	vegetable	train

2. Necesidad de Negocio

Como ya lo mencionamos en la sección del objetivo de este documento, la necesidad que tiene nuestro cliente, y por lo cual acuden a nuestro servicio, es que nosotros solventemos el poder lograr la identificación de áreas, construcciones o caminos que fueron capturadas en imágenes gracias a su infraestructura satelital y que, una vez teniendo una solución a esta necesidad, a ellos les sea más fácil trabajar en su toma de decisiones, inteligencia de negocio y con ello mejorar su colaboración con otras empresas o gobiernos.

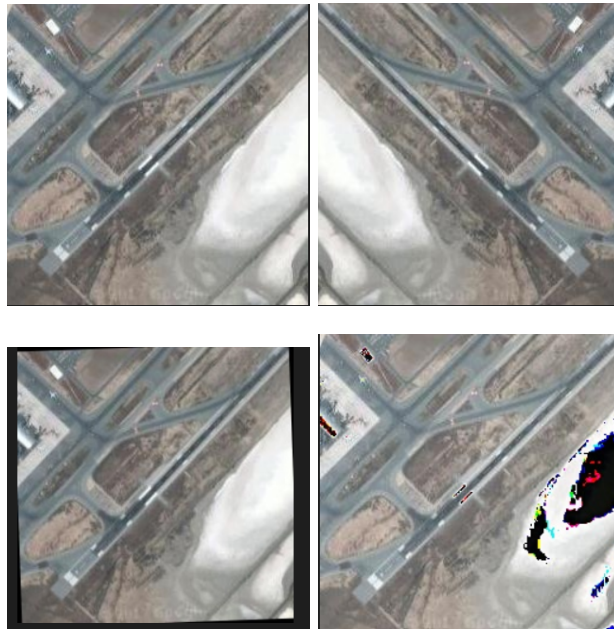
Es por lo cual que nuestros conocimientos en Machine learning y en especial en construcción de Redes Neuronales Convolucionales pueden ser de utilidad para realizar esta tarea.

3. Data Wrangling

3.1. Data Augmentation

La manipulación que realizaremos a los datos consiste en modificar composición o forma en la que se presenta cada imagen, en cuanto a rotar, hacer flip o cambiar la consistencia de los colores, esto con el objetivo de que el modelo cuente con distintas formas de "visualizar" las imágenes y con ello, a lo largo del aprendizaje, logre clasificar mejor.

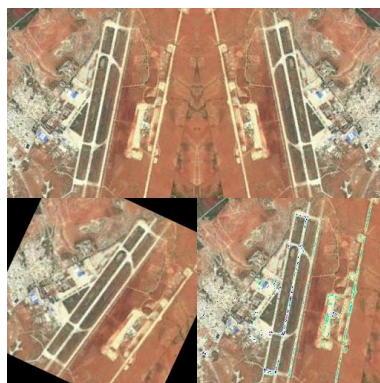
Para cada imagen haremos una rotación, un flip y un cambio en los canales de colores, con lo cual, para cada imagen tendremos 4 distintas versiones, a continuación mostramos un ejemplo:



3.2. Collage de imágenes

Con el fin de sintetizar la cantidad de datos que generamos a partir de Data Augmentation, lo que hacemos ahora es una especie de "collage" de cada imagen, es decir, que generaremos una nueva imagen que se compone de la imagen original, y las tres imágenes generadas en Data Augmentation (recordemos que este proceso solo se realizó para el conjunto de entrenamiento, pues para el conjunto de validación se espera que el modelo clasifique directamente la imagen).

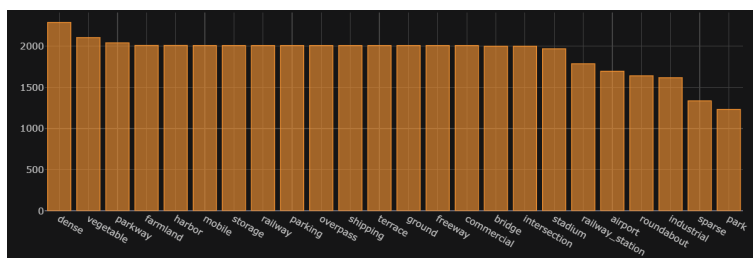
Con ello, la imagen de collage se ve así:



3.3. AEX

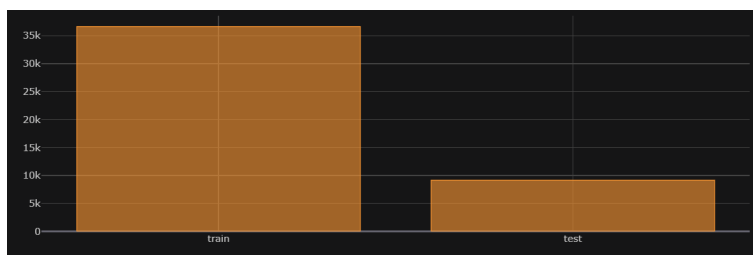
Una vez que tenemos las imágenes con las que trabajaremos para la puesta en marcha de nuestro modelo, así es la distribución de los datos.

Esta es la distribución de cada etiqueta, vemos está bastante balanceado, solamente tenemos un poco más de imágenes de Áreas Residenciales Densamente Pobladas, mientras que donde menos tenemos es en Parques y Áreas Residenciales Poco Pobladas.



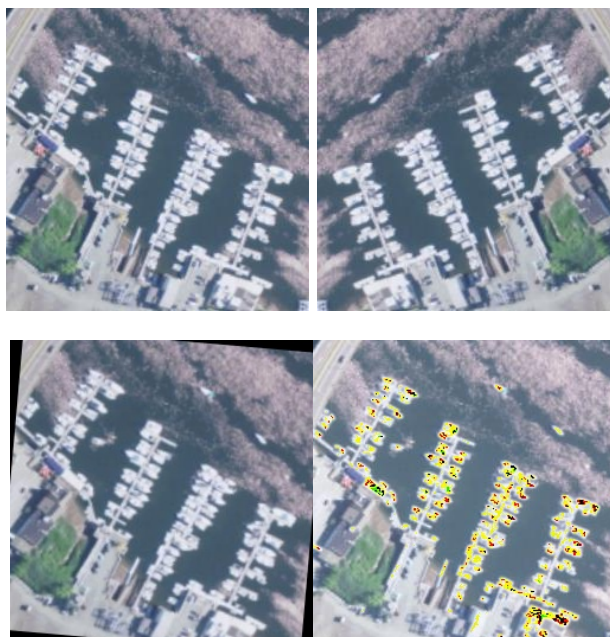
Cantidad de datos para entrenar y validar

Mientras que esta es la cantidad de imágenes que tenemos para entrenar son 36,642 y para validar son 9,161, recordemos que en train hay más imágenes debido al Data Augmentation realizado anteriormente, por separado hay 146,568 imágenes de enterramiento, son demasiadas imágenes y es por ello que recurrimos al Collage.



3.4. Visualización de las imágenes

Mostramos a continuación algunos ejemplos de las imágenes originales de los tipos de lugares con los que contamos, también mostramos sus respectivas versiones de Data Augmentation y su Collage. Ejemplo de un Harbor:

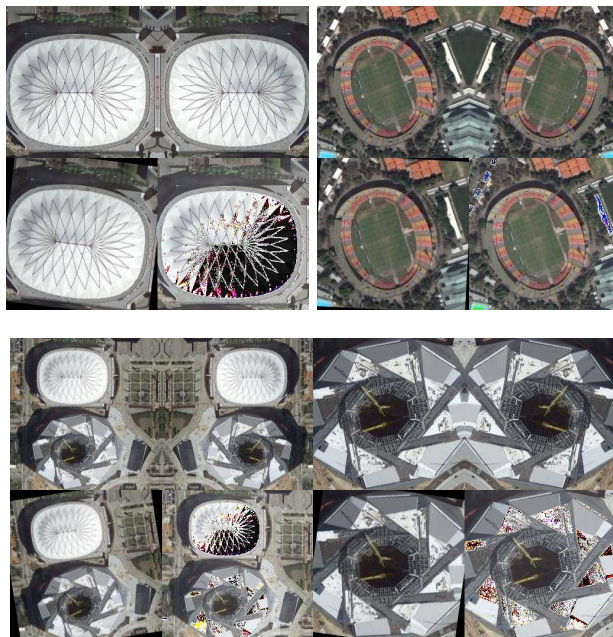


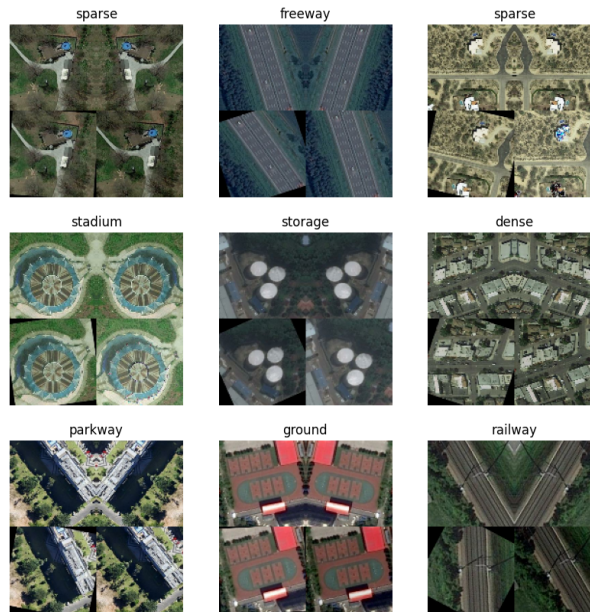
Ejemplo de un Distribuidor Vial:



Notamos que para colores blancos y negros es más notorio a simple vista el caso del cambio de composición de canales.

Finalmente, así se ve un collage de imágenes de Estadios:





4. Red Neuronal Convolutacional

4.1. Preproceso de Modelación

Explicaremos a continuación, cada etapa del proceso que requiere una Red Neuronal Convolutacional para configurar y construir las capas un modelo, para llegar hasta su entrenamiento y predicción de la clasificación de las imágenes, así como visualización de resultados.

4.1.1. Generadores

Los Generadores son relevantes en nuestro modelo ya que nos ayudarán a dar variabilidad y normalización a nuestros datos y manejar eficientemente el flujo de carga de los datos que reciba el entrenamiento y validación del modelo.

Con la clase *ImageDataGenerator* buscamos normalizar y aplicar un split al 20 % de los datos de conjunto de entrenamiento para lograr hacer una validación dentro de este mismo conjunto. Aprovechamos también para hacer un *vertical flip* y un zoom del 20 % a las imágenes, cosa que no aplicamos anteriormente en Data Augmentation.

Contaremos con 29,324 imágenes para train y 7,318 para validate para nuestras 24 labels.

A este generador le aplicamos ahora *flow_from_directory*, esto para cargar nuestras imágenes directamente desde nuestro directorio. Esta parte se aplica tanto para train, validation (el cual es el 20 % que se hizo de split en ImageDataGenerator), como para test.

En *flow_from_directory* también podemos redimensionar el tamaño de la imagen para sintetizar la información que proporcione cada imagen. En nuestro caso redimensionamos la imagen a (150,150) y, como hay tres canales de colores, la dimensión de cada imagen queda como (150,150,3).

En cuanto a *class_mode* esta será "categorical", ya que es una clasificación multiclase. Y por último definimos el *batch_size* a 72, es decir, que se procesarán las imágenes en lotes de 72 imágenes durante el entrenamiento.

4.1.2. Target

Como explicamos al inicio de este documento, tenemos 24 clases (lugares de las cuales tenemos imágenes), como buscamos categorizar las imágenes en estas clases, convertimos los labels a dígitos de 0 a 23, es decir, la categoría se representará como un número y no como una palabra.

4.2. Ejecución

4.2.1. Capas Convolucionales, Pooling y Dropout

Como inicio de construcción de nuestro modelo, lo que haremos será aplicar cinco capas convolucionales con 32, 64, 128 y dos de 256 filtros, todas con una *kernel.size* de (3,3), y todas con una función de activación *relu*", así mismo, aplicamos un *padding* tipo "same", es decir que la adición de ceros al rededor de la imagen mantenga el tamaño de esta.

Aplicamos un *pooling* del tipo máximo de tamaño (2,2), es decir, de la capa anterior se reducirá esta capa a un tamaño de 2*2 con cada entrada como el máximo de cada cuadro 2*2 de la capa anterior.

Finalmente aplicamos un Dropout a lo largo de las capas, de 0.2 y subiendo después a 0.25, para llegar a la capa de aplanado a 0.35, es decir, que en cada paso de capas se desactivarán aleatoriamente el 20 %, 25 % o 0.35 % de las neuronas, esto para evitar el sobreajuste.

4.2.2. Flatten y Capas densamente conectadas

Una vez que aplicamos todas nuestras capas convolucionales, Dropout y Pooling, la capa que se requiere ahora es la de *aplanado* o "Flatten", esta capa se compone de una hilera de en este caso 144 neuronas que

reúne la capa convolutacional anterior, como mencionamos, en esta capa habrá un Dropout de 0.3.

Finalmente, construimos la capa densamente conectada que se compone de 24 neuronas, una por cada *label* de nuestro conjunto de datos, cada salida contendrá la probabilidad de que la imagen en cuestión sea de cierta *label*, por lo que la neurona con la probabilidad más alta la tomaremos como lo que está prediciendo nuestro modelo.

Resumen de Modelo

Una vez realizado el proceso de selección de capas, así se ve el resumen de la configuración de nuestro modelo

Layer (type)	Output Shape	Param #
conv2d (conv2d)	(None, 16, 16, 3)	84
max_pooling2d (maxpooling2d)	(None, 8, 8, 3)	0
dropout (dropout)	(None, 8, 8, 3)	0
conv2d_1 (conv2d)	(None, 16, 16, 64)	16,448
max_pooling2d_1 (maxpooling2d)	(None, 8, 8, 64)	0
dropout_1 (dropout)	(None, 8, 8, 64)	0
conv2d_2 (conv2d)	(None, 16, 16, 128)	71,680
max_pooling2d_2 (maxpooling2d)	(None, 8, 8, 128)	0
dropout_2 (dropout)	(None, 8, 8, 128)	0
conv2d_3 (conv2d)	(None, 16, 16, 256)	289,792
max_pooling2d_3 (maxpooling2d)	(None, 8, 8, 256)	0
dropout_3 (dropout)	(None, 8, 8, 256)	0
conv2d_4 (conv2d)	(None, 8, 8, 256)	289,792
max_pooling2d_4 (maxpooling2d)	(None, 4, 4, 256)	0
dropout_4 (dropout)	(None, 4, 4, 256)	0
flatten (flatten)	(None, 1024)	0
dense (dense)	(None, 100)	211,700
dropout_5 (dropout)	(None, 100)	0
dense_1 (dense)	(None, 10)	1,400

Tenemos en total 1,313,896 parámetros a entrenar que componen el modelo, lo cual significa 5.01 MB de consumo de memoria.

4.2.3. Callbacks

Aplicamos los callbacks para detener el entrenamiento modelo según lo hayamos indicado, además de ir guardando el mejor resultado de cada época para así tener el mejor modelo posible al final de entrenamiento.

Después de haber hecho algunas pruebas al modelo, vimos que este no presenta que las métricas fueran muy constantes época tras época, por lo cual solo vemos solo necesario el uso del *EarlyStopping* con paciencia de 5 iteraciones para que, en caso de que no haya mucha diferencia de las métricas, este se detenga y nos dé ahí el resultado de la última época.

Así mismo, realizaremos un *ModelCheckpoint* para ir guardando los mejores desempeños de cada época (la cantidad de épocas que se aplicarán las mencionaremos más adelante).

4.3. Entrenamiento y Validación

Entrenamos el modelo con 30 épocas, las cuales son suficientes para que el modelo pueda lograr buenas puntuaciones para las métricas de desempeño (accuracy y loss).

En cuanto a tiempos de ejecución, en promedio cada época tardo en ejecutarse 12 minutos y con ello el modelo tardó en entrenar más de ¡tres horas!, lo cual muestra este tipo de algoritmos requieren de equipos con una gran capacidad de procesamiento.

RESULTADOS

En en el caso de accuracy obtuvimos 0.7530 y 0.7170 en entrenamiento y validación, mientras que para loss fue de 1.0718 y 1.2288 respectivamente, lo cual consideramos que es un desempeño bueno dada la cantidad de épocas con las que contamos.

```

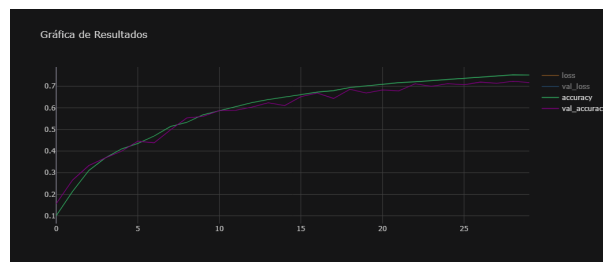
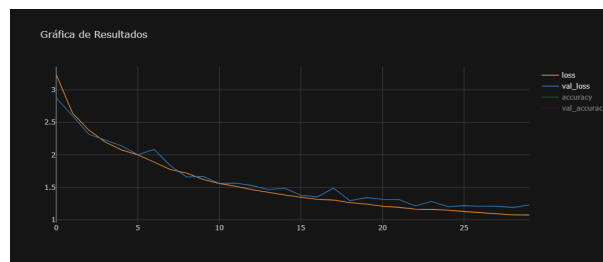
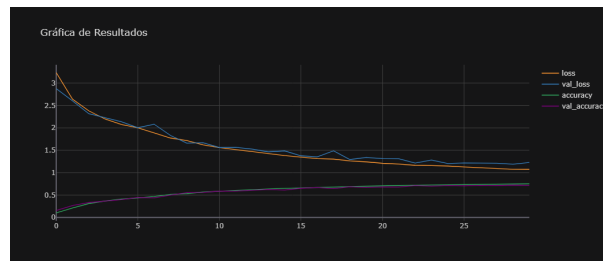
408/408 — 707s 2s/step - accuracy: 0.6626 - loss: 1.3390 - val_accuracy: 0.6521 - val_loss: 1.3775
Epoch 17/30
408/408 — 705s 2s/step - accuracy: 0.6711 - loss: 1.3175 - val_accuracy: 0.6689 - val_loss: 1.3527
Epoch 18/30
408/408 — 702s 2s/step - accuracy: 0.6726 - loss: 1.3217 - val_accuracy: 0.6436 - val_loss: 1.4871
Epoch 19/30
408/408 — 703s 2s/step - accuracy: 0.6895 - loss: 1.2795 - val_accuracy: 0.6858 - val_loss: 1.2935
Epoch 20/30
408/408 — 27832s 68s/step - accuracy: 0.7015 - loss: 1.2393 - val_accuracy: 0.6688 - val_loss: 1.34
Epoch 21/30
408/408 — 881s 2s/step - accuracy: 0.7054 - loss: 1.2162 - val_accuracy: 0.6827 - val_loss: 1.3155
Epoch 22/30
408/408 — 986s 2s/step - accuracy: 0.7168 - loss: 1.1883 - val_accuracy: 0.6787 - val_loss: 1.3126
Epoch 23/30
408/408 — 956s 2s/step - accuracy: 0.7170 - loss: 1.1801 - val_accuracy: 0.7114 - val_loss: 1.2141
Epoch 24/30
408/408 — 752s 2s/step - accuracy: 0.7321 - loss: 1.1422 - val_accuracy: 0.6996 - val_loss: 1.2817
Epoch 25/30
408/408 — 664s 2s/step - accuracy: 0.7280 - loss: 1.1496 - val_accuracy: 0.7121 - val_loss: 1.2041
Epoch 26/30
408/408 — 612s 1s/step - accuracy: 0.7348 - loss: 1.1392 - val_accuracy: 0.7070 - val_loss: 1.2167
Epoch 27/30
408/408 — 610s 1s/step - accuracy: 0.7426 - loss: 1.1122 - val_accuracy: 0.7193 - val_loss: 1.2096
Epoch 28/30
408/408 — 651s 2s/step - accuracy: 0.7469 - loss: 1.0882 - val_accuracy: 0.7134 - val_loss: 1.2103
Epoch 29/30
408/408 — 706s 2s/step - accuracy: 0.7527 - loss: 1.0758 - val_accuracy: 0.7229 - val_loss: 1.1901
Epoch 30/30
408/408 — 721s 2s/step - accuracy: 0.7530 - loss: 1.0718 - val_accuracy: 0.7170 - val_loss: 1.2288

```

Ahora bien, para observar el desempeño época a época hacemos uso del Callback de ModelCheckpoint para conocer el desempeño de cada época y con ello tener una mejor capacidad para analizar nuestros resultados.

4.4. Gráfica de Resultados

Con lo anterior, podemos ahora graficar los resultados de las métricas a lo largo de las 30 épocas que utilizamos. A continuación se muestran los resultados:



Podemos ver un buen comportamiento al pasar las épocas, teniendo como resultados finales un buen score en ambas métricas, las cuales nos puedan dar seguridad de que, de poder tener más iteraciones y mayor poder computacional, se logre una predicción aún más alta de la clasificación que se le va a dar a la imagen, ello nos lleva a que el modelo logre **generalizar** para datos nuevos y con ello se pueda dar una solución a la necesidad del cliente.

En importante notar que en la época 6 y 17 de pueden apreciar ciertos picos en el val_loss, volviendo a bajar en la siguiente iteración estados cerca del loss de train.

4.4.1. Predicción

Finalmente, ponemos a prueba nuestro modelo para buscar predecir imágenes que el modelo no vio previamente, en este caso son 9,161 imágenes nuevas, recordemos que estas imágenes no tienen el tratamiento de collage y data augmentation que nosotros aplicamos en el Data Wrangling, pues buscamos que se ponga a prueba el modelo sin modificaciones a la imagen y ver qué tan bien clasifica los lugares, por lo que es de esperarse que se aprecie una caída en el desempeño y con ello que las métricas sean más bajas.

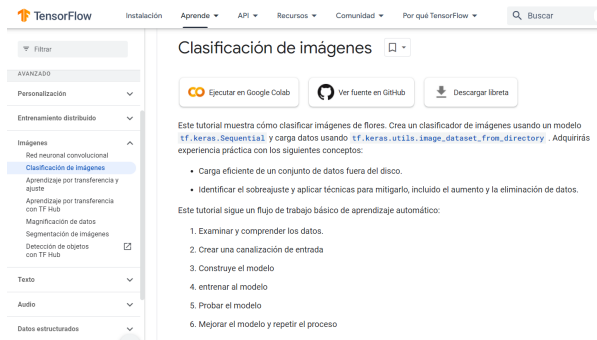
```
test_loss, test_acc = model.evaluate(test_generator)
print('Test loss: {} Test Acc: {}'.format(test_loss, test_acc))
✓ 42.8s
128/128 ————— 43s 333ms/step - accuracy: 0.5201 - loss: 2.2093
Test loss: 2.2380523681640625 Test Acc: 0.5180656909942627
```

5. Modelo Pre-Entrenado

Con el fin de tener una idea más clara de qué tan bien se desempeña nuestro modelo, aplicaremos un modelo que ya fue construido, el cual se puede encontrar en la página de Tensorflow, y se puede acceder a él por medio de este link:

<https://www.tensorflow.org/tutorials/images/classification?hl=es-419>

Este es un modelo de clasificación de imágenes, justo lo que nosotros hemos construido.



La constricción de este modelo cuenta con características muy similares con respecto a lo que nosotros realizamos para nuestro modelo, esto en cuanto a las capas convolucionales, las densamente conectadas, los generadores, data augmentation optimizadores y funciones de pérdida.

Sin embargo, comentamos que este modelo añade la particularidad del uso de **Optimizadores de flujo de datos** mediante `TF.DATA.AUTOTUNE` para manejar eficientemente la carga y procesamiento de imágenes.

```
AUTOTUNE = tf.data.AUTOTUNE

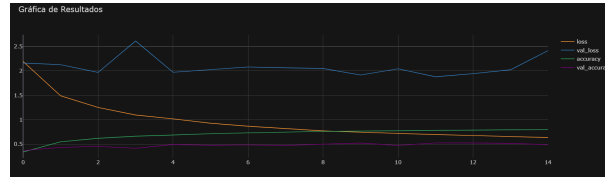
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

Este modelo no aplica Dropout entre capas, lo que puede alertarnos de que los resultados presentarán un sobreajuste.

Ahora bien, este modelo nos consume muchos menos recursos que el que nosotros construimos, como lo vemos en el siguiente *summary*, en el cual podemos ver que este modelo consume mejor memoria que el nuestro:

Layer (type)	Output Shape	Param #
rescaling_1 (Rescaling)	(None, 188, 188, 3)	0
conv2d_4 (conv2D)	(None, 188, 188, 16)	448
max_pooling2d_4 (MaxPooling2D)	(None, 90, 90, 16)	0
conv2d_5 (conv2D)	(None, 90, 90, 32)	4,640
max_pooling2d_5 (MaxPooling2D)	(None, 45, 45, 32)	0
conv2d_6 (conv2D)	(None, 45, 45, 64)	18,496
max_pooling2d_6 (MaxPooling2D)	(None, 22, 22, 64)	0
flatten_1 (Flatten)	(None, 30976)	0
dense_2 (Dense)	(None, 128)	3,965,056
dense_3 (Dense)	(None, 24)	3,096
Total params: 3,991,736 (15.23 MB)		
Trainable params: 3,991,736 (15.23 MB)		
Non-trainable params: 0 (0.00 B)		

Ahora, al aplicar el entrenamiento para 15 épocas vemos que, efectivamente como lo suponíamos, un fuerte sobreajuste, pues por ejemplo en la última época el accuracy de train fue de 0.799720, mientras que de validate fue de 0.488537, mientras que la pérdida fue 0.635238 y 2.415843 respectivamente. Veamos los resultados gráficamente y como se puede apreciar que si bien, hay un muy buen desempeño en el entrenamiento en loss yendo a la baja y accuracy, siendo contante a la alza, es todo lo contrario para la validación:



6. Comparación de Modelos de CNN

Con base en los resultados de las métricas de los modelos que pusimos a prueba y que mostramos en la sección anterior, vemos conveniente que el modelo que nosotros construimos sea puesto en producción para solventar la necesidad del cliente, ya que presenta muy buen comportamiento en las métricas de desempeño que pusimos a prueba.

En el caso del modelo de Tensorflow, al no aplicar una técnica de regularización, llámese Dropout, ese modelo tendrá un gran sobreajuste, pues en cada época, el modelo construirá la misma cantidad de características en cada capa y siempre trabajará con estas en la fase de validación.

7. Conclusiones del Proyecto

Gracias al análisis, exploración y manipulación de los datos con los que contamos, pudimos idear una forma de que con los datos que tenemos (el directorio de imágenes) se pudieran representar las mismas de una forma distinta, lo cual nos llevara a que, a la hora de poner a prueba nuestro modelo, este fuera capaz de predecir mejor.

En cuanto a la construcción del modelo, comentamos que este cuenta con una cantidad de capas, filtros y neuronas necesarios para que se logren buenos resultados. Finalmente, comparamos nuestro modelo con uno existente en la página de Tensorflow, para ver qué tal se desempeña este con nuestras imágenes y comparamos los resultados con los de nuestro modelo, vimos que este no se desempeña mejor que el nuestro, lo cual demuestra la capacidad que tiene nuestro modelo para trabajar correctamente.

Concluimos que se puede dar una solución a la necesidad del cliente para que con nuestro modelo de clasificación, logre clasificar más rápidamente el conjunto de imágenes con las que cuenta y con ello eficientar sus procesos y con ello generar valor para crecer como negocio. Para lograr asegurar los mejores resultados es importante contar con un equipo potente para que itere y realice una clasificación en todo el set de imágenes.